Code Explanation: Probability_Theory_and_Language_Model1.py [newhorizon...epoint.com]

This program demonstrates **Probability Theory in NLP** using **Unigram, Bigram, and Trigram Language Models**. It calculates the probability of the sentence:

> **"I love machine learning"** [newhorizon...epoint.com]

Step 1: Import Counter

```
from collections import Counter
```

Explanation

- Counter is a Python class used to count occurrences of items.
- It automatically stores frequencies.

Example:

```
Counter(['a','b','a'])
```

Output:

```
{'a':2, 'b':1}
```

Step 2: Create Sample Corpus

```
corpus = [
    "I love machine learning",
    "I love deep learning",
    "I study machine learning",
    "I study artificial intelligence",
    "Machine learning is interesting"
]
```

Explanation

A **corpus** is a collection of text documents or sentences.

The program uses 5 training sentences.

Corpus:

1. I love machine learning
2. I love deep learning

3. I study machine learning
4. I study artificial intelligence
5. Machine learning is interesting

These sentences are used to learn word probabilities. [newhorizon...epoint.com]

Step 3: Tokenization

```
1  tokenized_corpus = []
```

Creates an empty list.

```
1  for sentence in corpus:
```

Loops through every sentence.

```
1  words = sentence.lower().split()
```

Explanation

lower()

Converts text to lowercase.

Example:

```
1  "I Love NLP"
```

becomes

```
1  "i love nlp"
```

---

split()

Breaks the sentence into words.

Example:

```
1  "i love nlp".split()
```

Output:

```
1  ['i','love','nlp']
```

```
1  tokenized_corpus.append(words)
```

Adds the word list into the corpus.

Tokenized Corpus

```
[
 ['i','love','machine','learning'],
 ['i','love','deep','learning'],
 ['i','study','machine','learning'],
 ['i','study','artificial','intelligence'],
 ['machine','learning','is','interesting']
]
```

[newhorizon...epoint.com]

Step 4: Display Tokenized Corpus

```
print("Tokenized Corpus:")
```

Prints heading.

```
for sentence in tokenized_corpus:
    print(sentence)
```

Prints each tokenized sentence.

Output:

```
['i', 'love', 'machine', 'learning']
['i', 'love', 'deep', 'learning']
['i', 'study', 'machine', 'learning']
['i', 'study', 'artificial', 'intelligence']
['machine', 'learning', 'is', 'interesting']
```

[newhorizon...epoint.com]

Step 5: Count Unigrams

```
unigram_counts = Counter()
```

Creates empty unigram frequency table.

```
for sentence in tokenized_corpus:
    unigram_counts.update(sentence)
```

Counts every word.

Example:

```
i = 4
love = 2
machine = 3
learning = 4
```

```
total_words = sum(unigram_counts.values())
```

Calculates total number of words.

Corpus contains:

```
4+2+3+4+2+1+1+1+1+1
= 20 words
```

[newhorizon...epoint.com]

Step 6: Count Bigrams

```
bigram_counts = Counter()
```

Creates bigram dictionary.

```
for sentence in tokenized_corpus:
```

Processes each sentence.

```
for i in range(len(sentence)-1):
```

Moves through adjacent words.

```
bigram = (sentence[i], sentence[i+1])
```

Creates word pairs.

Examples:

```
('i','love')
('love','machine')
('machine','learning')
```

```
bigram_counts[bigram] += 1
```

Counts occurrences.

Example output:

```
('i','love') = 2
('i','study') = 2
('machine','learning') = 3
```

[newhorizon...epoint.com]

Step 7: Count Trigrams

```
trigram_counts = Counter()
```

Creates trigram frequency table.

```
for i in range(len(sentence)-2):
```

Moves through groups of three words.

```
trigram = (
    sentence[i],
    sentence[i+1],
    sentence[i+2]
)
```

Creates trigrams.

Examples:

```
('i','love','machine')
('love','machine','learning')
('i','study','machine')
```

```
trigram_counts[trigram] += 1
```

Counts frequencies.

[newhorizon...epoint.com]

Step 8: Test Sentence

```
test_sentence = "I love machine learning"
```

Sentence whose probability will be calculated.

```
words = test_sentence.lower().split()
```

Output:

```
['i','love','machine','learning']
```

[newhorizon...epoint.com]

Step 9: Unigram Probability

```
unigram_probability = 1.0
```

Initialize probability.

Formula:

For word **i**

```
probability = unigram_counts[word] / total_words
```

For **love**

For **machine**

For **learning**

```
unigram_probability *= probability
```

Final:

Example:

```
numerator = bigram_counts[(first, second)]
```

Gets pair frequency.

```
denominator = unigram_counts[first]
```

Gets first word frequency.

Formula:

Example:

Examples:

The code calculates each conditional probability and multiplies them together.

[newhorizon...epoint.com]

Step 12: Print Results

```
print("\nWord Counts:")
print(unigram_counts)
```

Displays unigram frequencies.

```
print("\nBigram Counts:")
print(bigram_counts)
```

Displays bigram frequencies.

```
print("\nTrigram Counts:")
print(trigram_counts)
```

Displays trigram frequencies.

```
print("\nSentence:", test_sentence)
```

Displays test sentence.

```
print("\nUNIGRAM PROBABILITY:", unigram_probability)
print("BIGRAM PROBABILITY:", bigram_probability)
print("TRIGRAM PROBABILITY:", trigram_probability)
```

Displays all calculated probabilities. [newhorizon...epoint.com]

NLP Concept Behind This Program

Model	Uses
Unigram	Considers each word independently
Bigram	Considers previous word
Trigram	Considers previous two words

Example:

Sentence:

```
I love machine learning
```

Unigram

Bigram

Trigram

As we move from **Unigram → Bigram → Trigram**, the language model uses more context, producing more accurate sentence probabilities. [newhorizon...epoint.com]